

Synthèse - LLMs Security & Cybesecurity

Introduction.

Ce cours propose une entrée structurée et opérationnelle à la sécurisation des systèmes fondés sur des grands modèles de langage (LLM), depuis la conception jusqu'à l'exploitation. Il articule trois référentiels complémentaires : la taxonomie de risques « OWASP Top 10 for LLMs/GenAI », qui formalise les vulnérabilités spécifiques aux applications à base de LLM (injections de prompt, fuites d'informations sensibles, gestion de sortie, supply-chain, etc.) et leurs contre-mesures prioritaires ([GenAI OWASP](#)) ; le cadre MITRE ATLAS, qui décrit les tactiques et techniques adverses propres à l'IA et permet de raisonner en chaînes d'attaque et d'ancrer les scénarios d'évaluation/red teaming ; et le NIST AI Risk Management Framework (AI RMF 1.0), utilisé comme enveloppe de gouvernance pour relier objectifs, contrôles et métriques tout au long du cycle de vie des modèles et des agents ([NIST](#)). Le fil directeur est constant : partir des usages (assistants, RAG, agents outillés), caractériser la surface d'attaque socio-technique, sélectionner des architectures et des garde-fous adaptés, puis en mesurer l'effet par des preuves observables (journaux, télémétrie, taux de blocage, coûts). L'objectif est de doter l'étudiant d'un cadre conceptuel solide et de réflexes expérimentaux transférables aux environnements de production.

Objectifs et portée

À la fin du cours, l'étudiant pourrait :

- expliquer les mécanismes d'attaque clés (injection directe/indirecte et "content smuggling", sur-permission des outils, exfiltration via API) ; 2) instrumenter une application LLM pour **mesurer** latence, coût, taux de blocage, FP/FN ; 3) sélectionner des contre-mesures robustes (deny-by-default sur les outils, schémas/validations I/O, minimisation de contexte, sandboxing, rate-limiting, surveillance SOC). L'accent est mis sur des preuves observables (traces, journaux, rejouabilité) et sur l'alignement avec le cycle **Govern-Map-Measure-Manage** du NIST AI RMF pour relier techniques et gouvernance. [Publications NIST](#)

Compétences visées

- **Cartographier les risques LLM01 → LLM10** et proposer des contrôles prioritaires.

- **Concevoir des contrats d'outils** (permissions minimales, pré/post-conditions, garde-fous de confirmation) et des pipelines de validation des sorties (JSON schema, allowlists, parsers stricts).
- **Exécuter un “AI red/blue teaming” léger** : scénariser des attaques avec ATLAS, rejouer, instrumenter, puis documenter l'efficacité défensive et les coûts.
- **Mettre en place une télémétrie SOC** adaptée aux LLM (détection d'injection/jailbreak, abus d'outils, DoS par prompts gourmands).

Architecture pédagogique

Six blocs progressifs structurent le semestre :

1. **Anatomie d'une appli LLM** (prompts, contexte, outils, exécution) ;
2. **Menaces OWASP LLM** avec démonstrations (injection directe/indirecte, exfil) ;
3. **Défense I/O** : validations de schéma, parsers, filtres, sandbox outillage ;
4. **Agency & plugins** : contrats, approbations humaines, limites d'autonomie ;
5. **ATLAS & red teaming** : narrer une chaîne d'attaque et l'instrumenter ;
6. **Gouvernance** : NIST AI RMF, métriques de risque, intégration SOC/IR. [Publications NIST](#)

Dispositif d'évaluation

- **Lab 0 (non noté)** : environnement local, journalisation, rejouabilité.
- **Lab 1** : prompt injection directe/indirecte + “content smuggling”, mesure FP/FN et coût ; atténuations (rejet hors-contrat, minimisation de contexte, confirmations).
- **Lab 2 : insecure output handling** : forcer le modèle à produire du HTML/SQL/CLI ; valider par schémas/allowlists et sandbox ; prouver la réduction de surface.
- **Lab 3 : poisoning & RAG** : empoisonner des documents, vérifier robustesse (vérifications hors-bande, provenance, filtrage) et coûts de mitigation.
- **Projet final (par 2–3)** : définir un cas d'usage, cartographier LLM01 → LLM10, déployer contrôles (contrats d'outils, validations I/O, sandbox, télémétrie), exécuter un mini red/blue teaming ATLAS et livrer **preuves** (traces, métriques, avant/après).

Environnement technique

Stack volontairement sobre et reproductible : Python, un LLM local/hosté, “guardrails” I/O (validateurs JSON), sandbox d'exécution, journaux structurés, et **intégration SOC** (alertes spécifiques à l'IA, détections jailbreak/injection/DoS). Les ateliers peuvent s'appuyer sur des ressources Microsoft Defender for Cloud pour les **détections gérées** des menaces IA, à fins de comparaison avec les contrôles maison.

Résultats attendus

Chaque étudiant apportera une analyse critique qui pourrait :

- **Démontrer** une attaque (ex. LLM01) et la bloquer proprement ;
 - **Justifier** ses choix par des métriques (latence, tokens, coût, FP/FN, impact UX) ;
 - **Documenter** une architecture défensive alignée NIST AI RMF ;
 - **Narrer** une chaîne d'attaque ATLAS et l'instrumenter de bout en bout (rejeu + preuves). Cette approche crée un langage commun entre équipes produit, sécurité et conformité.
- [Publications NIST](#)

Références principales pour le cours

- **OWASP Top 10 for LLM Applications** (dernier état, risques LLM01 → LLM10 et mitigations).
- **MITRE ATLAS** (tactiques/techniques d'attaques IA et scénarios de red teaming).
- **NIST AI RMF 1.0** (cadre de gouvernance Govern • Map • Measure • Manage). [Publications NIST](#)
- **Microsoft Defender for Cloud – contenus sécurité IA** (télémétrie, détections jailbreak/prompt injection/DoS).
- **RedTeam Arena** : [plateforme](#) de challenge pour le jailbreak des modèles LLMs.